C++ 线程安全的单例模式演变

Posted on 2023-09-03 12:45:39 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2Us7GguoWj4KlSVEMlK0maTaDET)

简介

本文介绍C++ 线程安全的单例模式如何实现，通过介绍单例模式的演变历程，给读者更完备的实现单例模式的方案。

局部静态变量

我们知道当一个函数中定义一个局部静态变量，那么这个局部静态变量只会初始化一次，就是在这个函数第一次调用的时候，以后无论调用几次这个函数，函数内的局部静态变量都不再初始化。
那我们可以利用局部静态变量这一特点实现单例

```
1. class Single2 {
2. private:
3.     Single2()
4.     {
5.     }
6.     Single2(const Single2&) = delete;
7.     Single2& operator=(const Single2&) = delete;
8. public:
9.     static Single2& GetInst()
10.    {
11.        static Single2 single;
12.        return single;
13.    }
14. };
```

上述版本的单例模式在C++11 以前存在多线程不安全的情况，编译器可能会初始化多个静态变量。
但是C++11推出以后，各厂商优化编译器，能保证线程安全。所以为了保证运行安全请确保

使用C++11以上的标准。

饿汉式初始化

在C++11 推出以前，局部静态变量的方式实现单例存在线程安全问题，所以部分人推出了一种方案，就是在主线程启动后，其他线程没有启动前，由主线程先初始化单例资源，这样其他线程获取的资源就不涉及重复初始化的情况了。

```
1. //饿汉式
2. class Single2Hungry
3. {
4. private:
5.     Single2Hungry()
6.     {
7.     }
8.     Single2Hungry(const Single2Hungry&) = delete;
9.     Single2Hungry& operator=(const Single2Hungry&) = delete;
10. public:
11.     static Single2Hungry* GetInst()
12.     {
13.         if (single == nullptr)
14.         {
15.             single = new Single2Hungry();
16.         }
17.         return single;
18.     }
19. private:
20.     static Single2Hungry* single;
21. };
```

调用如下

```
1. //饿汉式初始化
2. Single2Hungry* Single2Hungry::single = Single2Hungry::GetInst();
3. void thread_func_s2(int i)
4. {
5.     std::cout << "this is thread " << i << std::endl;
6.     std::cout << "inst is " << Single2Hungry::GetInst() << std::endl;
7. }
8. void test_single2hungry()
9. {
10.     std::cout << "s1 addr is " << Single2Hungry::GetInst() << std::endl;
11.     std::cout << "s2 addr is " << Single2Hungry::GetInst() << std::endl;
12.     for (int i = 0; i < 3; i++)
13.     {
14.         std::thread tid(thread_func_s2, i);
15.         tid.join();
16.     }
17. }
```

饿汉式是从使用角度规避多线程的安全问题，很多情况下我们很难从规则角度限制开发人员，所以这种方式不是很推荐。

懒汉式初始化

很多人觉得什么时候调用初始化是用户的权利，不应该加以限制，所以就有了懒汉式方式初始化资源，在用到时如果没有初始化单例则初始化，如果初始化了则直接使用.
所以这种方式我们要加锁，防止资源被重复初始化。

```
1. class SinglePointer
2. {
3. private:
4.     SinglePointer()
5.     {
6.     }
7.     SinglePointer(const SinglePointer&) = delete;
8.     SinglePointer& operator=(const SinglePointer&) = delete;
9. public:
10.     static SinglePointer* GetInst()
11.     {
12.         if (single != nullptr)
13.         {
14.             return single;
15.         }
16.         s_mutex.lock();
17.         if (single != nullptr)
18.         {
19.             s_mutex.unlock();
20.             return single;
21.         }
22.         single = new SinglePointer();
23.         s_mutex.unlock();
24.         return single;
25.     }
26. private:
27.     static SinglePointer* single;
28.     static std::mutex s_mutex;
29. };
```

调用如下

```
SinglePointer* SinglePointer::single = nullptr;
std::mutex SinglePointer::s_mutex;
void thread_func_lazy(int i)
{
    std::cout << "this is lazy thread " << i << std::endl;
    std::cout << "inst is " << SinglePointer::GetInst() << std::endl;
}
void test_singlelazy()
{
    for (int i = 0; i < 3; i++)
    {
        std::thread tid(thread_func_lazy, i);
        tid.join();
    }
    //何时释放new的对象？造成内存泄漏
}
```

这种方式存在一个很严重的问题，就是当多个线程都调用单例函数时，我们不确定资源是被哪个线程初始化的。
回收指针存在问题，存在多重释放或者不知道哪个指针释放的问题。

智能指针

我们能想到一个自动初始化资源并且自动释放的方式就是智能指针。利用智能指针自动回收资源。

```
1. //可以利用智能指针完成自动回收
2.
3. class SingleAuto
4. {
5. private:
6.     SingleAuto()
7.     {
8.     }
9.     SingleAuto(const SingleAuto&) = delete;
10.    SingleAuto& operator=(const SingleAuto&) = delete;
11. public:
12.    ~SingleAuto()
13.    {
14.        std::cout << "single auto delete success " << std::endl;
15.    }
16.    static std::shared_ptr<SingleAuto> GetInst()
17.    {
18.        if (single != nullptr)
19.        {
20.            return single;
21.        }
22.        s_mutex.lock();
23.        if (single != nullptr)
24.        {
25.            s_mutex.unlock();
26.            return single;
27.        }
28.        single = std::shared_ptr<SingleAuto>(new SingleAuto);
29.        s_mutex.unlock();
30.        return single;
31.    }
32. private:
33.    static std::shared_ptr<SingleAuto> single;
34.    static std::mutex s_mutex;
35. };
```

调用方式如下

```
std::shared_ptr<SingleAuto> SingleAuto::single = nullptr;
std::mutex SingleAuto::s_mutex;
void test_singleauto()
{
    auto sp1 = SingleAuto::GetInst();
    auto sp2 = SingleAuto::GetInst();
    std::cout << "sp1  is  " << sp1 << std::endl;
    std::cout << "sp2  is  " << sp2 << std::endl;
    //此时存在隐患，可以手动删除裸指针，造成崩溃
    // delete sp1.get();
}
```

这样开辟的资源交给智能指针管理免去了回收资源的麻烦。
但是有些人觉得虽然智能指针能自动回收内存，如果有开发人员手动 `delete` 指针怎么办?
所以有人提出了利用辅助类帮助智能指针释放资源，将智能指针的析构设置为私有。

```
1. //为了规避用户手动释放内存，可以提供一个辅助类帮忙回收内存
2. //并将单例类的析构函数写为私有
3.
4. class SingleAutoSafe;
5. class SafeDeletor
6. {
7. public:
8.     void operator()(SingleAutoSafe* sf)
9.     {
10.         std::cout << "this is safe deleter operator()" << std::endl;
11.         delete sf;
12.     }
13. };
14. class SingleAutoSafe
15. {
16. private:
17.     SingleAutoSafe() {}
18.     ~SingleAutoSafe()
19.     {
20.         std::cout << "this is single auto safe deletor" << std::endl;
21.     }
22.     SingleAutoSafe(const SingleAutoSafe&) = delete;
23.     SingleAutoSafe& operator=(const SingleAutoSafe&) = delete;
24.     //定义友元类，通过友元类调用该类析构函数
25.     friend class SafeDeletor;
26. public:
27.     static std::shared_ptr<SingleAutoSafe> GetInst()
28.     {
29.         //1处
30.         if (single != nullptr)
31.         {
32.             return single;
33.         }
34.         s_mutex.lock();
35.         //2处
36.         if (single != nullptr)
37.         {
38.             s_mutex.unlock();
39.             return single;
40.         }
41.         //额外指定删除器
42.         //3 处
```

```
43.         single = std::shared_ptr<SingleAutoSafe>(new SingleAutoSafe, SafeDeletor());
44.         //也可以指定删除函数
45.         // single = std::shared_ptr<SingleAutoSafe>(new SingleAutoSafe, SafeDelFunc);
46.         s_mutex.unlock();
47.         return single;
48.     }
49. private:
50.     static std::shared_ptr<SingleAutoSafe> single;
51.     static std::mutex s_mutex;
52. };
```

`SafeDeletor` 就是删除的辅助类，实现了仿函数。构造智能指针时指定了 `SafeDeletor` 对象，这样就能帮助智能指针释放了。

但是上面的代码存在危险，比如懒汉式的使用方式，当多个线程调用单例时，有一个线程加锁进入3处的逻辑。
其他的线程有的在1处，判断指针非空则跳过初始化直接使用单例的内存会存在问题。
主要原因在于SingleAutoSafe * temp = new SingleAutoSafe() 这个操作是由三部分组成的
1 调用allocate开辟内存
2 调用construct执行SingleAutoSafe的构造函数
3 调用赋值操作将地址赋值给temp

而现实中2和3的步骤可能颠倒，所以有可能在一些编译器中通过优化是1，3，2的调用顺序，
其他线程取到的指针就是非空，还没来的及调用构造函数就交给外部使用造成不可预知错误。
为解决这个问题，C++11 推出了std::call_once函数保证多个线程只执行一次

call_once

C++11 提出了 `call_once` 函数，我们可以配合一个局部的静态变量 `once_flag` 实现线程安全的初始化。
多线程调用 `call_once` 函数时，会判断 `once_flag` 是否被初始化，如没被初始化则进入初始化流程，调用我们提供的初始化函数。
但是同一时刻只有一个线程能进入这个初始化函数。

```
class SingletonOnce {
private:
    SingletonOnce() = default;
    SingletonOnce(const SingletonOnce&) = delete;
    SingletonOnce& operator = (const SingletonOnce& st) = delete;
    static std::shared_ptr<SingletonOnce> _instance;

public :
    static std::shared_ptr<SingletonOnce> GetInstance() {
        static std::once_flag s_flag;
        std::call_once(s_flag, [&]() {
            _instance = std::shared_ptr<SingletonOnce>(new SingletonOnce);
            });

        return _instance;
    }

    void PrintAddress() {
        std::cout << _instance.get() << std::endl;
    }

    ~SingletonOnce() {
        std::cout << "this is singleton destruct" << std::endl;
    }
};

std::shared_ptr<SingletonOnce> SingletonOnce::_instance = nullptr;
```

调用方式如下

```
void TestSingle() {

    std::thread t1([]() {
            std::this_thread::sleep_for(std::chrono::seconds(1));
            SingletonOnce::GetInstance()->PrintAddress();
        });

    std::thread t2([]() {
            std::this_thread::sleep_for(std::chrono::seconds(1));
            SingletonOnce::GetInstance()->PrintAddress();
    });

    t1.join();
    t2.join();
}
```

为了使用单例类更通用，比如项目中使用多个单例类，可以通过继承实现多个单例类

```cpp
//为了让单例更加通用，可以做成模板类
template <typename T>
class Singleton {
protected:
    Singleton() = default;
    Singleton(const Singleton<T>&) = delete;
    Singleton& operator=(const Singleton<T>& st) = delete;
    static std::shared_ptr<T> _instance;
public:
    static std::shared_ptr<T> GetInstance() {
        static std::once_flag s_flag;
        std::call_once(s_flag, [&]() {
            _instance = std::shared_ptr<T>(new T);
            });
        return _instance;
    }
    void PrintAddress() {
        std::cout << _instance.get() << std::endl;
    }
    ~Singleton() {
        std::cout << "this is singleton destruct" << std::endl;
    }
};
template <typename T>
std::shared_ptr<T> Singleton<T>::_instance = nullptr;
```

比如我们想实现单例类，就像我们之前在网络编程中介绍的那样，可以通过继承实现单例模式

```cpp
//想使用单例类，可以继承上面的模板，我们在网络编程中逻辑单例类用的就是这种方式
class LogicSystem :public Singleton<LogicSystem>
{
    friend class Singleton<LogicSystem>;
public:
    ~LogicSystem(){}
private:
    LogicSystem(){}
};
```

总结

如果你只是实现一个简单的单例类推荐使用返回局部静态变量的方式
如果想大规模实现多个单例类可以用call_once实现的模板类。

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

源码链接

https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(2762) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(562) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 👁浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作

不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站

个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)